CSV Document Querying Language Manual

Matthew Heath (mh6n23)

1. Language Syntax

1.1 Language Grammar

<program> ::= <operation>
<operation> ::= <quoted_string>
| “SELECT” “[“ <condition_list> “]” “(“ <operation> “)”
| “REPLACE” “[“ <replace_list> “]” “(“ <operation> “)”
| “REPLACE” “[“ <replace_list> “]” “WHERE” “[“ <condition_list> “]” “(“ <operation> “)”
| “PROJECT” “[“ <column_list> “]” “(“ <operation> “)”
| <join_type> “(“ <operation> “)” “ON” “[“ <condition_unit> “]” “(“ <operation> “)”
| “(“ <operation> “)” “X” “(“ <operation> “)”
| “(“ <operation> “)” “UNION” “(“ <operation> “)”
| “(“ <operation> “)” “-” “(“ <operation> “)”
| “SAVE” <quoted_string> “(“ <operation> “)”

<condition_list> ::= <condition_unit> | <condition_unit> <combinator> <condition_list>

<condition_unit> := <int> <operator> <int> | <int> <operator> <quoted_string>

<column_list> ::= <int> | <int> “,” <column_list>

<combinator> ::= “AND” | “OR”

<operator> ::= “==” | “!=”

<replace_list> ::= <replace_unit> | <replace_unit> “,” <replace_list>

<replace_unit> ::= <int> “->” <int> | <int> “->” <quoted_string>

<join_type> ::= “NaturalJoin” | “LeftJoin” | “SemiJoin”

<int> ::= [0-9]+

<quoted_string> ::= \”[^\”]*\”

1.2 Language Core Concept

Each program consists of a single outer operation. Each operation (apart from the single filename used for loading data, which is what the quoted string operation is used for) contains at least one inner operation which is evaluated first. The result of this inner operation is then passed to the outer operation for it to use. When there are no more outer operations to perform, the result of performing the operations will be automatically written to stdout in a CSV format.

1.3 Language Use Examples

Task Explanation	Language Input (.cql file contents)	CSV File Contents	Result
Getting the contents of a CSV file by referencing its name and outputting them	“A.csv”	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12	Printed Output: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12
Returning all the rows of a CSV file with the value “12” at index 2 (3rd column)	SELECT [2 == “12”] (“A.csv”)	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12	Printed Output: 1,Breath of the Wild,12 3,Metroid Prime,12
Returning all rows of a CSV file with the value “1” at index 0 or value “7” at index 2	SELECT [0 == “1” OR 2 == “7”] (“A.csv”)	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12	Printed Output: 1,Breath of the Wild,12 2,Super Mario Odyssey,7
Returning all rows of a CSV file with the value “Metroid Prime” at index 1 and value “12” at index 2	SELECT [1 == "Metroid Prime" AND 2 == "12"] ("A.csv")	A.csv 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12 4,Metroid Prime,16	Printed Output: 3,Metroid Prime,12
Replace each value at index 2 in each row of the csv file with	REPLACE [2 -> "3"] ("A.csv")	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12	Printed Output: 1,Breath of the Wild,3 2,Super Mario Odyssey,3 3,Metroid Prime,3
Replace the value at index 1 in each row of the csv file if it is “Breath of the Wild”	REPLACE [1 -> "Tears of the Kingdom"] WHERE [1 == "Breath of the Wild"] ("A.csv")	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12	Printed Output: 1,Tears of the Kingdom,12 2,Super Mario Odyssey,7 3,Metroid Prime,12
Replace each value at index 0 with “Hello” and each value at index 1 with “World”	REPLACE [0 -> "Hello", 1 -> "World"] ("A.csv")	A.csv: 1,Breath of the Wild,12	Printed Output: 1,Hello,World

		2,Super Mario Odyssey,7 3,Metroid Prime,12	2,Hello,World 3,Hello,World
Project (extract) the column at index 1	PROJECT [1] ("A.csv")	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12	Printed Output: Breath of the Wild Metroid Prime Super Mario Odyssey
Project (extract) the columns at indexes 0 and 1	PROJECT [0,1] ("A.csv")	A.csv: 1,Breath of the Wild 2,Super Mario Odyssey 3,Metroid Prime	Printed Output: 1,Breath of the Wild 2,Super Mario Odyssey 3,Metroid Prime
Perform a natural join on the contents of two CSV files where the value index 2 for file A.csv matches the value at index 0 for file B.csv (See 2.5 for how natural joins work in our language)	NaturalJoin ("A.csv") ON [2 == 0] ("B.csv")	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12 4,Luigi's Mansion,7 B.csv: 7,Suitable For All 12,Parental Guidance	Printed Output: 1,Breath of the Wild,12,12,Parental Guidance 2,Super Mario Odyssey,7,7,Suitable For All 3,Metroid Prime,12,12,Parental Guidance
Perform a left join on the contents of two CSV files where the value index 2 for file A.csv matches the value at index 0 for file B.csv	LeftJoin ("A.csv") ON [2 == 0] ("B.csv")	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12 4,Luigi's Mansion,7 B.csv: 7,Suitable For All 12,Parental Guidance	Printed Output: 1,Breath of the Wild,12,12,Parental Guidance 2,Super Mario Odyssey,7,7,Suitable For All 3,Metroid Prime,12,12,Parental Guidance 4,Call of Duty,18, ,
Perform a semi join on the contents of two CSV files where the value index 2 for file A.csv matches the value at index 0 for file B.csv	SemiJoin ("A.csv") ON [2 == 0] ("B.csv")	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12	Printed Output: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12

		4,Luigi's Mansion,7 B.csv: 7,Suitable For All 12,Parental Guidance	
Get the cartesian product of the contents of two csv files	(“A.csv”) X (“B.csv”)	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12 B.csv: line 1 line 2	Printed Output: 1,Breath of the Wild,12,line 1 1,Breath of the Wild,12,line 2 2,Super Mario Odyssey,7,line 1 2,Super Mario Odyssey,7,line 2 3,Metroid Prime,12,line 1 3,Metroid Prime,12,line 2
Get the union of the contents of two csv files	(“A.csv”) UNION (“B.csv”)	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12 B.csv: 4,Luigi’s Mansion,7 5,New Horizons, 3	Printed Output: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12 4,Luigi’s Mansion,7 5,New Horizons,3
Get the difference of the contents of two csv files	(“A.csv”) - (“B.csv”)	A.csv: 1,Breath of the Wild,12 2,Super Mario Odyssey,7 3,Metroid Prime,12 B.csv: 1,Breath of the Wild,12 3,Metroid Prime,12	Printed Output: 2,Super Mario Odyssey,7

2. Built in Commands/Functions (Essential)

2.1 Loading File Contents

"{file_name}.csv"

Referencing the name of a CSV file stored in the correct directory will load the contents of this file, which can then be used as the input for outer operations. Otherwise, just the contents of the file will be outputted. The file must have the ".csv" extension or an error will be shown. The use of a file name is the only way to terminate an operation.

2.2 Selecting Rows

SELECT [{condition_list}] ({operation})

The "SELECT" operation allows you to select all the rows from the inner operation that satisfy the conditions in the condition list. A condition list and individual condition unit are defined as follows:

{condition_list} ::= {condition_unit} {combinator} {condition_list} | {condition_unit}
{combinator} ::= AND | OR
{condition_unit} ::= {column1} {operator} {column2}
| {column} {operator} {quoted_string}
{combinator} ::= == | !=

Each condition is in the form of column1 equals (==) / doesn't equal (!=) column2 or column1 equals/doesn't equal a value. Each column is referenced with an integer, with 0 representing the first column. Conditions can then be combined using AND or OR. Condition1 AND Condition2 OR Condition3 will be evaluated as Condition1 AND (Condition2 OR Condition3) using left associativity.

2.3 Replacing Row Values

REPLACE [{replace_list}] ({operation})
REPLACE [{replace_list}] WHERE [{condition_list}] ({operation})

The "REPLACE" operation allows you to perform replacements on the inner operation according to the supplied replacement list which is defined as:

{replace_list} ::= {replace_unit} | {replace_unit} , {replace_list}
{replace_unit} ::= {column_to_be_replaced} -> {column_to_replace_with} |
{column_to_be_replaced } -> "{new_string}"

Each replacement uses the value on the left of the arrow to specify the column number whose value will be replaced, and the right of the arrow another column number to replace the value with or a brand-new string value which is enclosed in quotes. Multiple

replacements can be done in one operation by joining them together with commas. The "REPLACE" operation can optionally be used with the "WHERE" keyword and a condition list (See 2.2) so that the replacements are only performed on rows which satisfy the conditions.

2.4 Projecting Columns

PROJECT [{column_list}] ({operation})

The "PROJECT" operation will extract all the columns from the inner operation that are listed in the column list, which is defined as followed:

{column_list} ::= {column_number} , {column_list}

Each column number is separated by a comma. The columns will be arranged in the order specified, and there can be multiple copies of each column.

2.5 Natural Join

NaturalJoin ({operation_1}) ON [{condition_unit}] ({operation_2})
{condition_unit} ::= {operation_1_column} == {operation_2_column}

The "Natural Join" operation will perform a join on each row in inner operation 1 where its specified column is equal to the specified column of inner operation 2. If the column in a row in operation 1 doesn't match any of the rows in operation 2 using their respective columns, that row will not be included in the output. The condition unit used must be the "Equal Column" version.

2.6 Cartesian Product

{operation_1} X {operation_2}

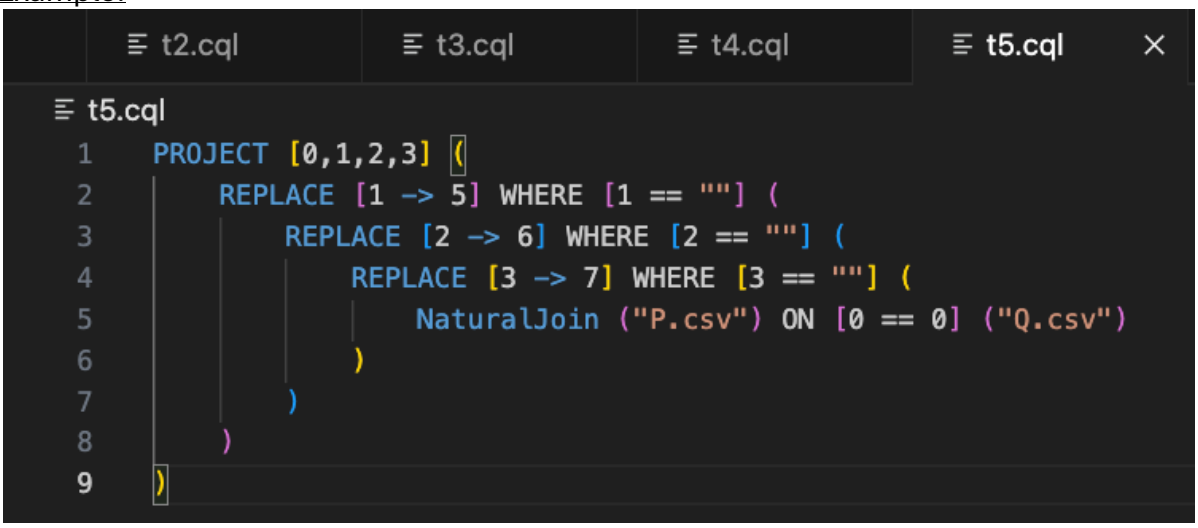
The "Cartesian Product" operation will output all the concatenated row combinations of inner operation 1 and inner operation 2, with inner operation 1's row contents being first in each row. It produces the set of all pairs (a,b), where a is a row from operation 1 and b is row from operation 2.

3. Extensions

3.1 Syntax Highlighting

We created a VSCode extension that would apply syntax highlighting to files with the ".cql" extension and then installed it onto the stack project. This highlights all the key words/symbols that somebody would use when developing a program in the language, such as "SELECT" or "REPLACE".

Example:



3.2 Error Messages

We added error messages within the interpreter using the error function in Haskell which will inform the user if there is a problem with their written program.

Error Cause	Error Message
A CSV filename entered didn't end in ".csv"	"Error: You entered a file name without the .csv extension"
A CSV file does not have a consistent arity	"Error: The CSV File " ++ fileName ++ " does not have the same arity on each row."
A CSV file has an invalid trailing new line	"Error: The CSV File " ++ fileName ++ " has an invalid trailing new line."
A union operation was attempted on two inner operations with different arities.	"Error: You are trying to perform a union on two sets of data with different arities."
A difference operation was attempted on two inner operations with different arities.	"Error: You are trying to perform a difference on two sets of data with different arities."
A join type was referenced that isn't supported by the language.	"Error: Invalid Join type"
An invalid word was used to join conditions (not AND or OR)	"You are using an invalid word to combine conditions: " ++ combinator

A condition references a column number which is out of bounds for an inner operation	"Error: You are checking a condition for Column Number " ++ show columnNumber1 ++ " which doesn't exist in row " ++ show row (message changes depending on which columns are invalid)
Unsupported operator used during condition	"Error: Invalid operator used within a condition"
A projection references a column number which is out of bounds for an inner operation	"Error: You are trying to project Column Number " ++ show columnNumber ++ " which doesn't exist in row " ++ show row
A join condition references a column number which is out of bounds for one of the inner operations	e.g. "Error: You are trying to perform a Natural Join using Column Number " ++ show column1 ++ " which doesn't exist in row " ++ show table1Row
A join condition is not in the correct format of column1 == column2	"Invalid join condition. Should be in the form of {columnNumber1} == {columnNumber2}"

3.3 Additional Commands/Functions

3.3.1 Left Join

LeftJoin ({operation_1}) ON [{condition_unit}] ({operation_2})

The “Left Join” operation will perform a left join on each row in inner operation 1 where its specified column is equal to the specified column of inner operation 2. If the column in a row in operation 1 doesn’t match any of the rows in operation 2, that row will have empty values attached to it to match the arity of the other rows.

3.3.2 Semi Join

SemiJoin ({operation_1}) ON [{condition_unit}] ({operation_2})

The “Semi Join” operation will perform a semi join on each row in inner operation 1, where it will select each row from operation 1 that would’ve been joined with a row from inner operation 2 where their specified column were equal.

3.3.3 Union

({operation_1}) UNION ({operation_2})

The “UNION” operation will append the rows from operation 2 onto the end of operation 1. Both operations must have tables of the same arity.

3.3.4 Difference

({operation_1}) - ({operation_2})

The “DIFFERENCE” operation will remove all the rows from operation 1 that are identical to a row in operation 2. It will then output only the remaining rows. Both operations must have tables of the same arity.

3.4 File Saving

SAVE “{file_name}.csv” ({operation})

The “SAVE” operation will save the contents of the inner operation to a new CSV file with the specified name in the correct format. It will then pass the result of the inner operation to the outer operation if there is one.

Example 1:

SAVE “B.csv” (“A.csv”)

A.csv:
apples,5
bananas,10
oranges,15

Output:
apples,5
bananas,10
oranges,15

Newly Created File B.csv:
apples,5
bananas,10
oranges,15

The SAVE operation takes the contents of the inner operation (which is just the contents of “A.csv”) and saves them to the specified file name “B.csv”. Since there is no outer function, the output is just the contents of “A.csv”.

Example 2:

PROJECT [0] (
SAVE “C.csv” (
PROJECT [0,0] (
SAVE “B.csv” (
“A.csv”))))

A.csv:
apples,5

bananas,10
oranges,15

Output:
apples
bananas
oranges

Newly Created File B.csv:
apples,5
bananas,10
oranges,15

Newly Created File C.csv:
apples,apples
bananas,bananas
oranges,oranges

The first SAVE operation takes the contents of the inner operation (which is just the contents of "A.csv") and saves them to the specified file name "B.csv". The inner operation result ("A.csv" contents) is then passed to the outer operation, which performs a projection of the first column twice. This is passed to another SAVE operation, which saves the result of this projection to "C.csv". Finally, the result of this projection is passed to an outer projection which just extracts the first column. Since there are no more outer operations, this is the final output.

3.5 Program Commenting

We allowed for comments to be used when writing programs in our language by configuring our Lexer so that any line starting with "—" is ignored.

Example: